Guía detallada de la implementación CI/CD para Azure Databricks

GitHub Actions, Terraform, seguridad y Declarative Automation Bundles

Documento didáctico del estado actual del repositorio

Proyecto:	`proyecto-databricks-cicd`
Alcance actual:	Entorno Dev preparado, sin recursos cloud creados
Fecha:	8 de octubre de 2026
Presupuesto objetivo:	S/200 mensuales

Este documento explica qué se creó, por qué existe cada pieza, cómo se validó y qué falta ejecutar.

Índice

1. Propósito y estado actual **3**
1.1. Qué está implementado en el repositorio . . . 3
1.2. Qué todavía no se ha ejecutado . . . 3
1.3. Qué se hace manualmente y qué automatiza Terraform . . . 3

2. Arquitectura y flujo general **3**
2.1. Por qué se promueve el mismo SHA . . . 4

3. Mapa del repositorio **4**

4. Python, entorno virtual y dependencias **5**
4.1. Qué es un entorno virtual . . . 5
4.2. Qué contiene pyproject.toml . . . 5

5. Validaciones de código y seguridad **5**
5.1. Pruebas implementadas . . . 6
5.2. Qué ocurrió durante la validación real . . . 6

6. Terraform desde cero **7**
6.1. Conceptos esenciales . . . 7
6.2. Ciclo de comandos . . . 7
6.3. Por qué existen dos capas . . . 7
6.4. State local y archivos ignorados . . . 7
6.5. Validación efectuada . . . 8

7. Infraestructura Azure definida **8**

8. Configuración Databricks definida **8**

9. GitHub Actions: validación del Pull Request **8**
9.1. Jobs de CI . . . 9
9.2. Por qué el PR no posee secretos . . . 9

10. GitHub Actions: despliegue Dev **9**
10.1. Secuencia exacta del job . . . 9

11. OIDC y mínimo privilegio **10**

12. Bundle, carpeta normal y smoke test **10**
12.1. Debugging manual en Dev y QA . . . 10

13. Flujo completo de un cambio **10**

14. Controles de costo **11**

15. Archivos de seguridad y reproducibilidad **11**

15.1. .gitignore 11

15.2. Lockfiles Terraform 11

15.3. Dependabot 11

16. Qué significa "listo" en este punto **11**

17. Glosario breve **12**

18. Próximo recorrido recomendado **13**

1. Propósito y estado actual

El proyecto es un laboratorio profesional y progresivo para aprender a llevar código Python, SQL y notebooks desde GitHub hasta Azure Databricks. La primera meta no es desplegar de inmediato en tres entornos, sino demostrar de punta a punta el flujo hacia **Dev**, controlar los costos y recién después incorporar QA y Prod.

1.1. Qué está implementado en el repositorio

- Estructura de código Python, SQL, notebooks y pruebas.
- Configuración reproducible de herramientas de calidad y seguridad.
- Terraform para crear el workspace Azure Databricks Dev y su presupuesto.
- Terraform para configurar identidad OIDC, catálogo, esquema y permisos del workspace.
- Un Bundle que publica una carpeta normal y administra el Job de smoke test.
- Un workflow de validación de Pull Requests.
- Un workflow de despliegue automático hacia Dev después del merge a `main`.
- Documentación operativa, reglas de contribución y registros de decisiones arquitectónicas.

1.2. Qué todavía no se ha ejecutado

- No existe todavía una suscripción Azure asociada al proyecto.
- No se ejecutó `terraform plan` con valores reales ni `terraform apply`.
- No se creó ningún workspace Databricks y no existe consumo cloud generado por el proyecto.
- No se creó todavía el repositorio remoto de GitHub ni el GitHub Environment `dev`.
- QA y Prod están diseñados conceptualmente, pero no se aprovisionan en esta fase.

Esta distinción es importante: **código de infraestructura** no equivale a **infraestructura creada**. Terraform sólo realiza cambios reales cuando se revisa y ejecuta un `apply` autorizado.

1.3. Qué se hace manualmente y qué automatiza Terraform

La persona propietaria crea la cuenta Microsoft/Azure, activa MFA, registra facturación, convierte la suscripción gratuita a Pay-As-You-Go, acepta términos, selecciona la suscripción mediante Azure CLI y autoriza cada `apply`. Son decisiones de identidad y facturación que no deben delegarse a un pipeline.

Después de ese bootstrap, Terraform crea el Resource Group, presupuesto, workspace Dev, identidad y permisos. No se deben crear manualmente esos recursos en el portal. Terraform tampoco despliega cada notebook: los Bundles administran el código y Jobs con una cadencia mucho mayor que la infraestructura.

2. Arquitectura y flujo general

El flujo elegido combina integración continua sobre `main` con promoción de commits inmutables. `main` representa código integrado; no representa automáticamente la versión de Producción.

```
feature/* o fix/*
       |
       | Pull Request: calidad, pruebas y seguridad
       v
     main ---- despliegue automatico ----> Dev
       |
       | tag vX.Y.Z-rc.N
       v
      QA ---- pruebas + aprobacion ----> mismo SHA
       |
       | tag final vX.Y.Z
       v
     Prod
```

Si un release candidate falla en QA, Prod continúa usando la versión estable anterior. La corrección se hace en una nueva rama `fix/*`, se integra mediante otro PR y genera un nuevo SHA y un nuevo candidato. No se mueve ni se reutiliza el tag fallido.

2.1. Por qué se promueve el mismo SHA

Un **SHA** identifica de forma única un commit Git. Promover el mismo SHA reduce la pregunta "¿QA probó exactamente lo que llegó a Prod?" a una comparación verificable. No se recompila ni se edita el código durante la promoción.

3. Mapa del repositorio

Ruta	Responsabilidad
`src/databricks_cicd/`	Código Python reutilizable y contrato de configuración por entorno.
`tests/`	Pruebas unitarias ejecutadas con Pytest.
`sql/`	SQL versionado y validado con dialecto Databricks.
`proyecto_test/`	Árbol de notebooks y configuración reflejado en el workspace.
`resources/`	Jobs y otros recursos declarados por el Bundle.
`databricks.yml`	Targets, rutas y variables del Bundle.
`scripts/`	Utilidades del pipeline, como validación de variables requeridas.
`infra/azure/dev/`	Recursos Azure: presupuesto, resource group y workspace Dev.
`infra/databricks/dev/`	Configuración interna de Databricks después de crear el workspace.
`infra/modules/`	Módulos Terraform reutilizables.
`.github/workflows/`	CI de Pull Requests y despliegue Dev.
`docs/adr/`	Decisiones de arquitectura y sus motivos.
`pyproject.toml`	Metadatos Python, dependencias de desarrollo y configuración de herramientas.
`.gitignore`	Evita versionar state, variables locales, entornos virtuales y credenciales.

4. Python, entorno virtual y dependencias

4.1. Qué es un entorno virtual

Un **entorno virtual** o `venv` es una carpeta aislada que contiene un intérprete de Python y sus paquetes. Evita mezclar dependencias del proyecto con paquetes globales del equipo.

Durante la implementación se creó localmente `.venv/` y se instalaron las dependencias de desarrollo declaradas en `pyproject.toml`. La carpeta está ignorada por Git porque:

- puede reconstruirse a partir de la configuración;
- cambia entre Windows, Linux y macOS;
- versionarla añadiría miles de archivos ajenos al código del proyecto.

El proceso conceptual fue:

```
python -m venv .venv
.venv/Scripts/python -m pip install --editable ".[dev]"
```

`-editable` instala el proyecto apuntando a la carpeta de trabajo. Así, los cambios en `src/` se reflejan sin reinstalar el paquete. El extra `[dev]` instala únicamente las herramientas necesarias para desarrollar y validar.

4.2. Qué contiene pyproject.toml

`pyproject.toml` centraliza:

- nombre, versión y versión mínima de Python;
- sistema de construcción basado en Setuptools;
- dependencias de desarrollo;
- configuración de Pytest, Ruff y Bandit;
- ubicación del paquete dentro de `src/`.

Las dependencias de validación incluyen Ruff, Pytest, SQLFluff, Yamllint, Bandit, `detect-secrets` y `pip-audit`. No son servicios cloud: son programas ejecutados en el equipo local o en un runner de GitHub Actions.

5. Validaciones de código y seguridad

Herramienta	Qué valida	Qué evita
Ruff	Estilo, imports, errores Python y formato.	Código inconsistente, imports sin usar y defectos simples.
Pytest	Comportamiento esperado mediante pruebas.	Cambios que rompen reglas ya verificadas.
SQLFluff	Sintaxis y estilo SQL con dialecto Databricks.	SQL difícil de revisar o incompatible con las reglas elegidas.
Yamllint	Estructura y estilo de archivos YAML.	Workflows mal indentados o configuración ambigua.

Herramienta	Qué valida	Qué evita
Bandit	Análisis estático de seguridad para Python.	Patrones inseguros conocidos dentro del código.
detect-secrets	Patrones y entropía compatibles con secretos.	Tokens o claves enviados accidentalmente a Git.
pip-audit	Vulnerabilidades publicadas de dependencias.	Usar versiones con CVE o avisos de seguridad conocidos.
Terraform validate	Sintaxis, tipos y relaciones de configuración.	Planes inválidos antes de contactar Azure.

5.1. Pruebas implementadas

El módulo `config.py` modela tres valores obligatorios: entorno, catálogo y esquema. Las pruebas verifican que:

1. una configuración Dev válida se normalice correctamente;
2. una variable ausente cause un error explícito;
3. un entorno distinto de Dev, QA o Prod sea rechazado.

Este patrón mueve errores de configuración hacia el inicio del pipeline. Es preferible fallar antes de autenticarse o modificar Databricks que descubrir el problema a mitad del despliegue.

5.2. Qué ocurrió durante la validación real

La primera ejecución no fue artificialmente perfecta. Encontró:

- imports Python sin normalizar;
- archivos que Ruff deseaba reformatear;
- Yamllint intentando revisar el contenido de `.venv`;
- un salto de línea SQL incorrecto;
- versiones de `pip`, Pytest y SQLFluff reportadas con vulnerabilidades conocidas.

Las correcciones fueron: ordenar y formatear Python, excluir `.venv` del lint YAML, corregir el SQL, actualizar los rangos de dependencias y repetir todas las validaciones. El resultado final fue:

- tres pruebas unitarias aprobadas;
- Ruff, SQLFluff y Yamllint aprobados;
- Bandit sin incidencias;
- detección de secretos aprobada;
- `pip-audit` sin vulnerabilidades conocidas en las dependencias auditables.

Esto ilustra el propósito de CI: convertir problemas reproducibles en feedback antes del merge.

6. Terraform desde cero

6.1. Conceptos esenciales

Provider
Plugin que traduce Terraform a la API de una plataforma. Aquí se usan AzureRM y Databricks.

Resource
Objeto que Terraform crea o administra, por ejemplo un resource group o catálogo.

Variable
Entrada configurable; evita codificar IDs o valores de entorno dentro de los recursos.

Output
Dato producido por una capa para uso humano o para configurar la siguiente.

Module
Conjunto reutilizable de recursos con entradas y salidas.

State
Registro que vincula la configuración con recursos reales. Puede contener datos sensibles.

Lockfile
Archivo que fija las versiones exactas de providers y sus hashes.

6.2. Ciclo de comandos

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
```

`init` prepara providers; `fmt` revisa formato; `validate` comprueba la configuración; `plan` muestra cambios propuestos; `apply` ejecuta exactamente el plan. Sólo los dos últimos requieren valores y permisos cloud reales, y sólo `apply` crea o modifica recursos.

6.3. Por qué existen dos capas

Capa Azure: crea el contenedor cloud y el workspace. Incluye el presupuesto y etiquetas de costos. Usa el provider AzureRM.

Capa Databricks: se ejecuta cuando el workspace ya responde. Crea la identidad Databricks, política OIDC, permisos, catálogo, esquema, carpeta del proyecto y ruta protegida para el estado del Bundle. Usa providers Databricks a nivel de cuenta y workspace.

Separarlas evita intentar configurar una API que todavía no existe y permite destruir primero los objetos internos, y luego el workspace.

6.4. State local y archivos ignorados

En esta fase el state comienza local para facilitar el aprendizaje. Los patrones `*.tfstate`, `*.tfvars` y `*.tfplan` están ignorados. Los ejemplos `terraform.tfvars.example` sí se versionan porque sólo contienen marcadores ficticios.

Antes de añadir QA, el state se migrará a Azure Storage con cifrado, control de acceso, versionado y bloqueo. El state nunca debe pegarse en tickets ni enviarse por correo.

6.5. Validación efectuada

Se descargó temporalmente Terraform 1.13.4 con verificación SHA-256 para validar compatibilidad. Se ejecutaron `fmt`, `init -backend=false` y `validate` sobre ambas capas. Los providers quedaron fijados por lockfiles. El workflow de CI solicita Terraform 1.16.5.

`-backend=false` significa que la validación no intentó configurar state remoto. No se ejecutó `plan` ni `apply`; por tanto, no hubo cambios en Azure.

7. Infraestructura Azure definida

La capa Azure declara:

- un resource group para Dev;
- un workspace Azure Databricks Premium;
- un managed resource group con nombre controlado;
- etiquetas `project`, `environment`, `owner` y `managed_by`;
- un presupuesto mensual con alertas al 50, 75, 90 y 100 por ciento.

El módulo restringe intencionalmente `environment` a `dev`. Esto hace que intentar crear QA o Prod en la fase equivocada falle durante la validación.

Azure Budgets alerta, pero no corta el consumo. Además, el importe está expresado en la moneda de facturación de la suscripción, no necesariamente en soles. Antes del primer `apply` se debe convertir S/200 con margen conservador y confirmar la región.

8. Configuración Databricks definida

La segunda capa declara:

- un service principal exclusivo para CI/CD Dev;
- asignación del principal al workspace con rol de usuario, no administrador;
- una política de federación OIDC limitada al repositorio y GitHub Environment `dev`;
- la carpeta normal `/Workspace/data/proyecto_test`;
- la ruta protegida `/Workspace/.cicd/proyecto_test/dev`;
- `CAN_MANAGE` para CI/CD y edición temporal para desarrolladores en Dev;
- catálogo `proyecto_dev` y esquema `cicd`;
- permisos de datos específicos para la identidad del pipeline.

El catálogo y el esquema usan `force_destroy = false`. Si contienen datos, Terraform debe negarse a eliminarlos automáticamente. Es una barrera contra destrucción accidental.

9. GitHub Actions: validación del Pull Request

El workflow `validate-pr.yml` se dispara ante un PR hacia `main` o manualmente. Declara `contents: read` y no recibe credenciales cloud.

9.1. Jobs de CI

1. **Python, SQL and YAML**: instala el proyecto y ejecuta Ruff, SQLFluff, Yamllint y Pytest.
2. **Security checks**: ejecuta detección de secretos, Bandit y `pip-audit`.
3. **Terraform static validation**: revisa formato, inicializa sin backend y valida ambas capas.

Los jobs usan `timeout-minutes`; la concurrencia cancela validaciones antiguas del mismo PR. Las acciones principales están fijadas a SHA completos con un comentario que indica la versión legible.

9.2. Por qué el PR no posee secretos

Un contribuidor podría modificar el código que ejecuta CI. Entregar credenciales a ese proceso permitiría exfiltrarlas. Por eso la validación de PR sólo analiza código y no usa `id-token: write`, tokens Databricks ni secretos de Azure.

10. GitHub Actions: despliegue Dev

`deploy-dev.yml` se ejecuta después de un push a `main`, normalmente producido por un merge aprobado. Usa el GitHub Environment `dev` y declara sólo:

```
permissions:
  contents: read
  id-token: write
```

`id-token: write` no da permisos generales de escritura en GitHub. Permite pedir un token OIDC temporal que Databricks validará contra la política configurada.

10.1. Secuencia exacta del job

1. Descarga exactamente `github.sha`.
2. Valida variables obligatorias sin imprimir sus valores.
3. Instala la CLI oficial con `databricks/setup-cli`.
4. Comprueba la identidad OIDC mediante `databricks current-user me`.
5. Valida y despliega el Bundle parametrizado con `github.sha`.
6. Reconcilia rutas, tipos y hashes contra un manifiesto del repositorio.
7. Elimina archivos manuales y restaura contenido modificado o eliminado.
8. Ejecuta el Job serverless de humo declarado por el Bundle.
9. Escribe un resumen con commit, ruta y entorno, sin secretos.

La concurrencia `databricks-dev` impide dos despliegues simultáneos. El Bundle conserva una identidad de despliegue y una segunda ejecución del mismo SHA debe producir el mismo estado.

11. OIDC y mínimo privilegio

OIDC permite que GitHub demuestre ante Databricks qué repositorio y environment ejecutan el job. Databricks intercambia esa prueba por una credencial temporal. No existe un PAT de larga duración guardado en GitHub.

La política acepta un subject similar a:

```
repo:OWNER/REPOSITORY:environment:dev
```

Si el repositorio se renombra, transfiere o intenta usar otro environment, la autenticación falla. Es un control positivo, no un inconveniente.

El service principal no es workspace admin. Sólo recibe acceso al workspace, a la carpeta del proyecto, al estado del Bundle y al catálogo necesarios. QA y Prod tendrán identidades diferentes.

12. Bundle, carpeta normal y smoke test

El árbol local `proyecto_test/` se publica bajo `/Workspace/data/proyecto_test`. No aparece como Git Folder. El archivo `databricks.yml` define targets y rutas; `resources/` define Jobs. El estado técnico vive fuera de la carpeta editable, bajo `/Workspace/.cicd`.

Un manifiesto calcula la ruta remota, tipo y SHA-256 de cada archivo. Después de desplegar, el pipeline elimina objetos desconocidos y compara el contenido exportado. Sus guardas rechazan rutas amplias como `/Workspace` o `/Workspace/data`; sólo puede limpiar la carpeta exacta del proyecto.

El smoke test es deliberadamente pequeño:

```
result = spark.sql("SELECT 1 AS pipeline_is_healthy").first()
assert result.pipeline_is_healthy == 1
```

No prueba toda la lógica del proyecto. Confirma que autenticación, Bundle, reconciliación, lectura del notebook y cómputo serverless funcionan juntos. Las pruebas de negocio crecerán después sin usar datos reales.

12.1. Debugging manual en Dev y QA

Los desarrolladores pueden modificar archivos y notebooks en Dev y QA para diagnosticar. Esos cambios son temporales: deben copiarse a una rama y pasar por PR. El siguiente despliegue restaura lo que existe en Git, recrea eliminados y borra agregados manualmente. No existe sincronización automática workspace a Git. Prod permanece sin edición manual para desarrolladores.

13. Flujo completo de un cambio

1. El desarrollador crea `feature/descripcion` desde `main`.
2. Trabaja localmente dentro de `.venv` y ejecuta linters y pruebas.
3. Abre un PR a `main`.
4. GitHub ejecuta calidad, seguridad y Terraform sin credenciales cloud.
5. Una revisión humana aprueba y se realiza squash merge.
6. El push resultante a `main` inicia el workflow Dev.

7. OIDC entrega una identidad temporal de mínimo privilegio.
8. Databricks despliega el Bundle y reconcilia la carpeta contra el SHA.
9. Se ejecuta el smoke test serverless.
10. El resultado y el SHA quedan registrados en GitHub Actions.

En la fase futura, un tag `v0.1.0-rc.1` seleccionará un SHA para QA. Si pasa, el tag final `v0.1.0` apuntará al mismo SHA y será elegible para Prod.

14. Controles de costo

La estrategia para acercarse a S/200 mensuales es:

- crear sólo Dev al inicio;
- usar serverless o cómputo efímero;
- no mantener clústeres clásicos permanentes;
- revisar costos y forecast después de cada práctica;
- usar alertas progresivas;
- destruir el laboratorio si aparece gasto inesperado;
- crear QA y Prod sólo cuando exista una prueba que justifique su costo.

El orden seguro de destrucción es configuración Databricks primero y capa Azure después. Siempre se revisa `terraform plan -destroy` antes de aplicar la destrucción.

15. Archivos de seguridad y reproducibilidad

15.1. .gitignore

Excluye `.venv`, caches, archivos `.env`, certificados, `terraform.tfvars`, planes y states. Mantiene versionados los ejemplos sin credenciales.

15.2. Lockfiles Terraform

Los archivos `.terraform.lock.hcl` registran provider, versión seleccionada y hashes. Sí deben versionarse: permiten que CI y otros equipos utilicen la misma versión validada.

15.3. Dependabot

Revisa semanalmente dependencias Python, GitHub Actions y providers Terraform. Una actualización no se aplica directamente: genera un PR que debe pasar por los mismos checks.

16. Qué significa “listo” en este punto

Listo localmente

- código y configuración escritos;
- pruebas y linters aprobados;
- dependencias auditadas;
- Terraform inicializado y validado sin backend;
- documentación y ADRs disponibles.

Pendiente de credenciales y autorización humana

- alta Pay-As-You-Go y MFA;
- elección de región y conversión del presupuesto a la moneda de facturación;
- creación del remoto público GitHub;
- primer plan Azure con valores reales;
- revisión conjunta y primer apply;
- configuración de variables del GitHub Environment Dev;
- primera ejecución real del pipeline.

17. Glosario breve

CI
Integración continua: validación automática de cada cambio.

CD
Entrega o despliegue continuo: promoción automatizada y controlada.

Runner
Máquina efímera donde GitHub ejecuta un job.

Workflow
Archivo YAML que define eventos y jobs de GitHub Actions.

Environment
Frontera GitHub para variables, secretos y aprobaciones de un entorno.

Service principal
Identidad no humana usada por automatización.

OIDC
Federación que proporciona credenciales temporales sin secreto persistente.

SAST
Análisis estático de seguridad del código.

SCA
Análisis de vulnerabilidades de dependencias.

Idempotencia
Repetir una operación produce el mismo estado, sin duplicados.

Smoke test
Prueba pequeña que confirma que el camino crítico funciona.

Unity Catalog
Capa de gobierno y permisos para datos y objetos Databricks.

Release candidate
Versión candidata para QA, todavía no productiva.

18. Próximo recorrido recomendado

1. Leer `docs/01-azure-account-and-cost.md` y crear la suscripción personalmente.
2. Confirmar moneda de facturación, presupuesto equivalente y región disponible.
3. Crear el repositorio GitHub público y publicar el commit inicial.
4. Instalar Azure CLI y Terraform desde fuentes oficiales.
5. Completar `terraform.tfvars` local sin versionarlo.
6. Ejecutar juntos `terraform plan`; explicar cada alta, cambio o destrucción.
7. Sólo después de revisar el plan, autorizar `terraform apply` para Dev.
8. Configurar Databricks y GitHub Environment Dev.
9. Abrir un PR de prueba y observar cada job.
10. Medir costo real antes de decidir la creación de QA.

Principio rector: entender cada cambio antes de automatizarlo.